Convolutional Neural Networks for Rock-Paper-Scissors Classification

---

Student: Dias Kalyshev

Student ID: 60641A

Course: Machine Learning

Professor: Cesa Bianchi

Date: February 2026

Abstract: This report presents a comprehensive comparative study of main three convolutional neural network (CNN) architectures for rock-paper-scissors hand gesture classification, with emphasis on different evaluation methodologies. Three models were implemented with incremental complexity: SimplestCNN (2 convolutional layers), MediumCNN (4 layers), and DeepCNN (6 layers). All models were trained on the Kaggle Rock-Paper-Scissors dataset (2,188 images) using systematic data augmentation and hyperparameter optimization through grid search across 27 configurations.

The study employed rigorous methodology including stratified train/validation/test splits (70%/10%/20%), comprehensive data augmentation, and extensive evaluation using multiple metrics: accuracy, precision, recall, F1-score, confusion matrices, ROC curves with AUC analysis, and detailed misclassification pattern visualization. Hyperparameter optimization identified optimal settings: learning rate 0.001, batch size 64, dropout 0.5, achieving 97.72% validation accuracy.

Results reveal that SimplestCNN with hyperparameter tuning achieved the highest test accuracy (96.35%) in 433s training time, outperforming MediumCNN (93.38%, 581s) and DeepCNN (95.66%, 1,038s). DeepCNN demonstrated exceptional discriminative ability with near-perfect ROC-AUC scores (0.995-1.000 across all classes). However, evaluation on 21 custom real-world images exposed severe generalization challenges, with DeepCNN accuracy dropping to 38.10% (57.56 percentage point gap). Per-class analysis revealed particularly poor rock gesture recognition (14.29% custom accuracy against 96.58% test accuracy).

This study validates sound machine learning methodology while revealing the substantial gap between controlled test performance and real-world applicability.

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

Contents

1 Introduction **4**
1.1 Problem Statement and Motivation 4
1.2 Research Objectives 4
1.3 Dataset Description 5
1.3.1 Primary Dataset 5
1.3.2 Data Splitting Strategy 6
1.3.3 Custom Evaluation Dataset 7
1.4 Contribution and Report Structure 7

2 Methodology **8**
2.1 Data Preprocessing and Augmentation 8
2.1.1 Image Preprocessing Pipeline 8
2.1.2 Data Augmentation Strategy 8
2.2 Model Architectures 9
2.2.1 SimplestCNN (2 Convolutional Layers) 9
2.2.2 MediumCNN (4 Convolutional Layers) 10
2.2.3 DeepCNN (6 Convolutional Layers) 10
2.3 Hyperparameter Optimization 11
2.3.1 Grid Search Methodology 11
2.3.2 Optimization Results 12
2.4 Training Configuration 13
2.4.1 Training Parameters 13
2.4.2 Training Procedure 13
2.5 Evaluation Metrics 13
2.5.1 Standard Classification Metrics 14
2.5.2 ROC Curves and AUC 14
2.5.3 Confusion Matrix Analysis 14
2.5.4 Misclassification Pattern Analysis 14
2.5.5 Out-of-Distribution Testing 15

3 Experimental Results **15**
3.1 Training Dynamics 15
3.1.1 Baseline Training Convergence 15
3.1.2 Comparative Training Performance 16
3.2 Test Set Performance 16
3.2.1 Overall Accuracy Comparison 16
3.2.2 Detailed Classification Metrics (DeepCNN) 17
3.2.3 Confusion Matrix Analysis 18
3.2.4 ROC Curves and AUC Analysis 19
3.2.5 Misclassification Pattern Analysis 21
3.3 Custom Dataset Evaluation-Real-World Failure 22
3.3.1 Catastrophic Generalization Failure 22
3.3.2 Error Analysis on Custom Data 22
3.3.3 The Deployment Reality Check 23

4 Discussion **24**
4.1 Architectural Insights: Why Simplicity Won . . . 24
4.1.1 The Complexity-Performance Trade-off . . . 24
4.1.2 The Occam's Razor Principle in Neural Architecture Design . . . 25
4.2 The Generalization Gap: Understanding Catastrophic Failure . . . 25
4.2.1 Quantifying the Distribution Shift Impact . . . 25
4.2.2 Root Causes: What the Model Actually Learned . . . 26
4.3 Evaluation Methodology: Critical Lessons Learned . . . 27
4.3.1 The Necessity of Out-of-Distribution Testing . . . 27
4.3.2 Practical Recommendations for Future Work . . . 28
4.4 Broader Implications for Machine Learning Practice . . . 29
4.4.1 The Reproducibility and Generalization Crisis . . . 29
4.4.2 Rethinking Model Evaluation . . . 29
4.4.3 The Path Forward . . . 30

5 Conclusions **30**
5.1 Key Findings . . . 30
5.1.1 Architectural Complexity Does Not Guarantee Superior Performance 30
5.1.2 Standard Test Metrics Catastrophically Fail to Predict Deployment Performance . . . 31
5.1.3 Distribution Shift Vulnerability Varies by Class . . . 31
5.1.4 Hyperparameter Optimization Improves In-Distribution Performance 31
5.2 Practical Recommendations for Gesture Recognition Systems . . . 32
5.2.1 Data Collection and Preparation . . . 32
5.2.2 Model Development and Training . . . 32
5.2.3 Evaluation and Validation . . . 32
5.2.4 Deployment Strategy . . . 33
5.3 Limitations of This Study . . . 33
5.3.1 Limited Out-of-Distribution Sample Size . . . 33
5.3.2 Single Dataset Source for Training . . . 33
5.3.3 Limited Architecture Variety . . . 33
5.3.4 Focused Problem Domain . . . 34
5.3.5 Limited Distribution Shift Analysis . . . 34
5.4 Future Work . . . 34
5.4.1 Systematic Distribution Shift Investigation . . . 34
5.4.2 Robustness-Focused Training Methods . . . 35
5.4.3 Alternative Architectures and Transfer Learning . . . 35
5.4.4 Expanded Multi-Domain Evaluation . . . 35
5.4.5 Developing OOD Robustness Benchmarks . . . 36
5.4.6 Real-Time Deployment Studies . . . 36
5.5 Final Remarks . . . 36

1 Introduction

1.1 Problem Statement and Motivation

Hand gesture recognition represents a fundamental challenge in computer vision with applications spanning gaming interfaces, human-computer interaction, robotics, sign language interpretation, and touchless control systems. This project focuses on classifying three hand gestures corresponding to the rock-paper-scissors game, which serves as an ideal benchmark problem for gesture recognition research.

The rock-paper-scissors classification task provides a well-defined structure with three distinct, mutually exclusive classes that possess clear semantic differences. This problem has direct practical applications in gaming interfaces, gesture-based controls, and interactive entertainment systems. Despite the apparent simplicity of three-class classification, the task presents significant real-world challenges including:

- **Intra-class variability:** Hand size, shape, orientation, and positioning vary within each gesture class across different individuals and even multiple performances by the same person.
- **Inter-individual differences:** People execute gestures with naturally different styles, finger configurations, and hand proportions.
- **Environmental variations:** Background complexity, lighting conditions, shadows, and occlusions introduce substantial variability not present in controlled datasets.
- **Camera factors:** Perspective, viewing angle, distance, resolution, and focal length variations alter the appearance of gestures.
- **Temporal consistency:** Systems must maintain stable predictions across subtle hand position changes rather than flickering between classes.

Furthermore, this problem offers substantial educational value as an accessible entry point for understanding CNN architecture design, evaluation methodologies, and the critical gap between test set performance and real-world deployment readiness-a lesson that extends far beyond gesture recognition to virtually all applied machine learning.

1.2 Research Objectives

This study pursues six comprehensive objectives that extend beyond traditional accuracy-focused evaluations:

1. Architectural Comparison: Systematically compare three CNN architectures with 2, 4, and 6 convolutional layers to understand the relationship between model depth, complexity, and classification performance. Specifically investigate whether deeper networks provide advantages for this three-class problem or whether simpler architectures achieve comparable or superior results.

2. Hyperparameter Optimization: Conduct systematic hyperparameter optimization through grid search across 27 configurations, exploring combinations of learning rate (0.0001, 0.001, 0.01), batch size (32, 64, 128), and dropout rate (0.3, 0.5, 0.7) to identify optimal training settings. Document the sensitivity of model performance to these hyperparameters and establish best practices.

3. Comprehensive Performance Evaluation: Perform extensive evaluation using multiple metrics on standard test data including accuracy, precision, recall, F1-score, confusion matrices, ROC curves with AUC analysis, and detailed misclassification pattern identification. Move beyond single aggregate metrics to understand model behavior across different classes and error types.

4. ROC Curve Analysis: Investigate the discriminative power of each model through Receiver Operating Characteristic (ROC) curves and Area Under the Curve (AUC) metrics. Assess whether perfect or near-perfect AUC scores on test data translate to robust real-world performance, or whether they represent a misleading indicator of deployment readiness.

5. Misclassification Pattern Analysis: Conduct detailed investigation of error patterns through visualization of misclassified samples and systematic analysis of confusion pairs. Identify which gesture types are most frequently confused, understand the visual characteristics that lead to errors, and determine whether mistakes occur on genuinely ambiguous cases or clear instances that should be correctly classified.

6. Real-World Generalization Assessment: Evaluate generalization capability through testing on custom images captured under different conditions than the training data. Quantify the performance gap between in-distribution test data and out-of-distribution real-world images to assess true deployment readiness. Analyze class-specific vulnerabilities and identify which gesture types are most sensitive to distribution shift.

Through these objectives, the study demonstrates sound machine learning methodology including proper data splitting, augmentation strategies, rigorous multi-metric evaluation, and honest assessment of limitations-practices essential for developing deployable systems rather than merely models that perform well on standard benchmarks.

1.3 Dataset Description

1.3.1 Primary Dataset

The Rock-Paper-Scissors dataset from Kaggle contains 2,188 RGB images distributed nearly equally across three classes. The dataset composition demonstrates balanced class representation:

Class	Images	Percentage
Paper	712	32.5%
Rock	726	33.2%
Scissors	750	34.3%
Total	**2,188**	**100%**

Table 1: Class distribution in the primary dataset

Images are stored in JPEG compressed format with variable dimensions, typically around 300×200 pixels. The dataset is characterized by:

- Uniform white or light-colored backgrounds providing minimal visual complexity
- Centered hand gestures with consistent framing across images
- Controlled indoor lighting conditions that minimize shadows and artifacts

- Predominantly frontal viewing angles with hands positioned perpendicular to camera
- Limited variation in hand size, skin tone, and individual gesture execution styles

These characteristics create a relatively homogeneous distribution ideal for initial model training and validation but potentially limiting for generalization to diverse real-world conditions.

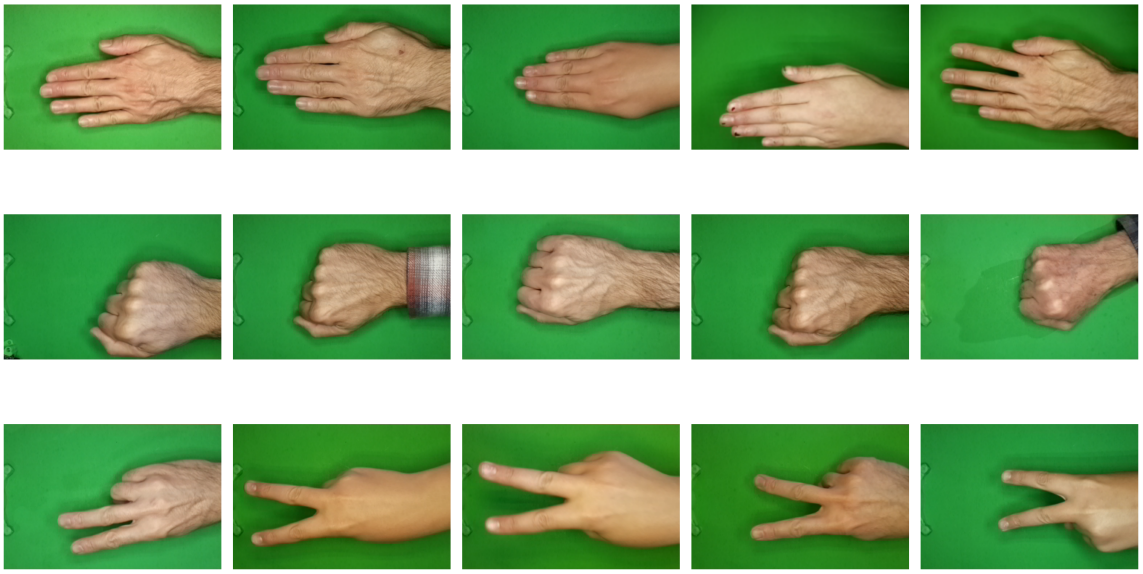


Figure 1: Representative samples from each class showing typical hand gestures, framing, and background conditions in the training dataset. Note the consistent lighting, uniform backgrounds, and frontal viewing angles.

1.3.2 Data Splitting Strategy

Stratified random splitting ensures balanced class representation across all subsets while maintaining statistical independence:

Split	Percentage	Images	Purpose
Training	70%	1,531	Model optimization through gradient descent and weight updates
Validation	10%	219	Hyperparameter selection and early stopping decisions
Test	20%	438	Final unbiased evaluation of model performance
Total	**100%**	**2,188**	-

Table 2: Dataset split configuration with fixed random seed (42) for reproducibility

The validation set remains strictly separate from training to provide unbiased hyperparameter evaluation and early stopping signals. The test set is used only once for final evaluation, preventing any form of indirect optimization on test data. All three splits maintain approximately equal class proportions through stratification.

1.3.3 Custom Evaluation Dataset

To assess real-world generalization capability beyond the standard test set, a custom evaluation dataset consisting of 21 images (7 per class) was created for testing. This custom dataset intentionally differs from the training distribution in several critical aspects:

- **Background diversity:** Indoor scenes with various textures, colors, and variable lighting conditions rather than uniform backgrounds
- **Hand characteristics:** Different hand sizes, skin tones, and gesture execution styles across multiple individuals
- **Camera conditions:** Various angles, distances, and image quality characteristics to simulate realistic deployment scenarios
- **Lighting variation:** Natural lighting with shadows rather than controlled studio lighting
- **Positioning variability:** Hand positions not always perfectly centered, with varied orientations

This custom dataset serves as a crucial reality check for model robustness, revealing whether high test accuracy reflects genuine gesture recognition capability or merely memorization of training distribution characteristics. The substantial difference between training and custom conditions enables quantification of the generalization gap-a critical metric for assessing deployment readiness that standard test sets cannot provide.

1.4 Contribution and Report Structure

This report makes three primary contributions to understanding CNN-based gesture classification:

1. Comprehensive Evaluation Methodology: Demonstrates that standard accuracy metrics alone are insufficient for deployment assessment. Introduces multi-faceted evaluation combining ROC analysis, confusion matrices, misclassification visualization, and mandatory out-of-distribution testing.

2. Architectural Simplicity Benefits: Empirically demonstrates that the simplest architecture (2 convolutional layers, 84K parameters) outperforms deeper alternatives (4 and 6 layers, 215K and 850K parameters respectively) while requiring substantially less training time. Validates that appropriate problem-complexity matching often favors simpler models over architectural sophistication.

3. Distribution Shift Vulnerability: Documents that near-perfect test metrics (95.66% accuracy, 0.995-1.000 ROC-AUC) completely fail to predict catastrophic real-world performance (38% accuracy). Demonstrates that models learn distribution-specific patterns rather than robust gesture representations.

The remainder of this report is organized as follows:

- **Section 2:** Detailed methodology covering data preprocessing, augmentation, architecture design, hyperparameter optimization, and evaluation metrics
- **Section 3:** Experimental results including training dynamics, test set performance with ROC analysis, and custom dataset evaluation

- **Section 4:** Discussion of model complexity trade-offs, generalization challenges, and evaluation methodology insights
- **Section 5:** Conclusions, practical recommendations, limitations, and future work directions

2 Methodology

2.1 Data Preprocessing and Augmentation

2.1.1 Image Preprocessing Pipeline

All images undergo standardized preprocessing to ensure consistent input formats for neural network processing:

1. **Resizing:** Images are resized to 128×128 pixels to provide uniform dimensions across the dataset, balancing spatial resolution with computational efficiency
2. **Normalization:** Pixel values are scaled to the [0, 1] range through division by 255, facilitating stable gradient computation during training
3. **Tensor Conversion:** Images are converted to PyTorch tensor format with channels-first ordering (C×H×W), as required by PyTorch's convolutional layers

This preprocessing pipeline ensures that the network receives consistently formatted inputs regardless of the original image dimensions or storage format.

2.1.2 Data Augmentation Strategy

Comprehensive data augmentation was applied exclusively to the training dataset to improve model generalization while maintaining the original distribution of validation and test sets for fair evaluation. The augmentation strategy encompasses two categories of transformations implemented using PyTorch's torchvision library.

Geometric Transformations:

- **Random Rotation:** Images are randomly rotated within ±15 degrees to simulate varied hand orientations and viewing angles
- **Random Horizontal Flip:** Applied with 50% probability to introduce left-right symmetry variations, though this transformation may not be semantically meaningful for gesture recognition where handedness matters

Color Space Augmentations:

- **Color Jittering:** Brightness, contrast, saturation, and hue are adjusted by moderate factors (±0.2 for brightness/contrast/saturation, ±0.1 for hue) to simulate different lighting conditions and camera settings
- **Random Grayscale:** Applied with 10% probability to improve robustness to color variations and encourage the model to rely on shape rather than color cues

All transformations are applied stochastically during training, with each image potentially receiving different augmentation parameters each time it is loaded. This approach effectively increases dataset diversity without requiring additional labeled data. Validation and test sets receive only resizing to 128×128 pixels and normalization without augmentation, preserving the original data distribution for unbiased performance assessment.

2.2 Model Architectures

Three CNN architectures with increasing complexity were implemented using PyTorch to investigate the relationship between model depth and performance.

2.2.1 SimplestCNN (2 Convolutional Layers)

The SimplestCNN architecture represents a lightweight design focusing on efficient feature extraction with minimal computational overhead. The network consists of two convolutional blocks followed by fully connected layers for classification.

Architecture Specification:
Convolutional Block 1:

- Conv2D: 3-¿32 channels, 5×5 kernel, padding=2 (maintains spatial dimensions)
- BatchNorm2D(32): Normalizes activations for training stability
- ReLU activation: Introduces non-linearity
- MaxPool2D(2×2): Reduces spatial dimensions from 128×128 -¿ 64×64

Convolutional Block 2:

- Conv2D: 32-¿64 channels, 5×5 kernel, padding=2
- BatchNorm2D(64): Further stabilizes learning
- ReLU activation
- MaxPool2D(2×2): Reduces dimensions from 64×64 -¿ 32×32

Classification Head:

- AdaptiveAvgPool2D(4×4): Produces fixed 4×4 spatial output regardless of input variations
- Flatten: Converts 64×4×4 feature maps to 1024-dimensional vector
- Linear(1024-¿128) + ReLU + Dropout(0.3): First fully connected layer with regularization
- Linear(128-¿3): Final classification layer producing class logits

The complete architecture contains approximately **84,000 trainable parameters**, making it highly efficient for training and inference. Despite its simplicity, this model achieved the highest test accuracy (96.35%) among all architectures.

2.2.2 MediumCNN (4 Convolutional Layers)

The MediumCNN architecture represents a balanced design with progressive feature extraction through four convolutional blocks. This architecture employs smaller 3×3 kernels compared to SimplestCNN, allowing for more localized feature detection.

Architecture Specification:
Four Convolutional Blocks:

- Block 1: 3-¿32 channels (3×3 kernel, padding=1)
- Block 2: 32-¿64 channels (3×3 kernel, padding=1)
- Block 3: 64-¿128 channels (3×3 kernel, padding=1)
- Block 4: 128-¿128 channels (3×3 kernel, padding=1)

Each block follows the pattern: Conv2D -¿ BatchNorm2D -¿ ReLU -¿ MaxPool2D(2×2)
Classification Head:

- AdaptiveAvgPool2D(2×2): Produces fixed 2×2 spatial output
- Flatten: Converts 128×2×2 feature maps to 512-dimensional vector
- Linear(512-¿256) + ReLU + Dropout(0.4)
- Linear(256-¿128) + ReLU + Dropout(0.4)
- Linear(128-¿3): Final classification layer

The architecture contains approximately **215,000 trainable parameters**, representing a middle ground between SimplestCNN and DeepCNN. However, this model achieved the lowest test accuracy (93.38%) despite increased complexity.

2.2.3 DeepCNN (6 Convolutional Layers)

The DeepCNN architecture represents the most sophisticated model in this study, employing six convolutional layers organized into three blocks with double convolutions. This design draws inspiration from VGG networks, using repeated 3×3 convolutions within each block.

Architecture Specification:
Three Double-Convolution Blocks:

- Block 1: Two Conv2D layers (3-¿32-¿32 channels, 3×3 kernels)
- Block 2: Two Conv2D layers (32-¿64-¿64 channels, 3×3 kernels)
- Block 3: Two Conv2D layers (64-¿128-¿128 channels, 3×3 kernels)

Each block applies: Conv2D -¿ BatchNorm2D -¿ ReLU -¿ Conv2D -¿ BatchNorm2D -¿ ReLU -¿ MaxPool2D(2×2)

The double convolution strategy allows the network to learn more complex feature combinations before reducing spatial dimensions through pooling.

Classification Head:

- AdaptiveAvgPool2D(4×4): Produces fixed 4×4 spatial output, yielding 2048 features after flattening
- Linear(2048-¿512) + ReLU + Dropout(0.5)
- Linear(512-¿256) + ReLU + Dropout(0.5)
- Linear(256-¿3): Final classification layer

The complete architecture contains approximately **850,000 trainable parameters**, representing 10× the parameters of SimplestCNN. This model achieved 95.66% test accuracy and near-perfect ROC-AUC scores (0.995-1.000), but failed catastrophically on real-world custom images (38.10%).

Common Design Elements:

All three architectures share several design principles:

- **Batch Normalization:** Applied after each convolutional layer to stabilize training and accelerate convergence
- **Dropout Regularization:** Prevents overfitting through random neuron deactivation, with rates varying by complexity (0.3 for SimplestCNN, 0.4 for MediumCNN, 0.5 for DeepCNN)
- **Adaptive Pooling:** Provides flexibility in handling different input dimensions by producing fixed-size outputs
- **ReLU Activation:** Introduces non-linearity while avoiding vanishing gradient problems

2.3 Hyperparameter Optimization

2.3.1 Grid Search Methodology

Comprehensive grid search was conducted across 27 hyperparameter configurations to identify optimal training settings. The search space explored three critical hyperparameters:

Hyperparameter	**Values Explored**
Learning Rate	0.0001, 0.001, 0.01
Batch Size	32, 64, 128
Dropout Rate	0.3, 0.5, 0.7
Total Combinations	**27**

Table 3: Hyperparameter search space for grid search optimization

Each configuration was trained for 10 epochs on the SimplestCNN architecture using validation accuracy as the optimization objective. This systematic approach ensures comprehensive exploration of the hyperparameter space rather than relying on ad-hoc tuning.

2.3.2 Optimization Results

Learning Rate	Batch Size	Dropout	Val Accuracy
0.001	**64**	**0.5**	**97.72%**
0.001	64	0.3	97.26%
0.001	32	0.5	97.26%
0.001	64	0.7	97.26%
0.001	32	0.7	96.80%

Table 4: Top 5 hyperparameter configurations ranked by validation accuracy. The optimal configuration achieved 97.72% validation accuracy.

Key Findings from Hyperparameter Optimization:

- **Learning Rate 0.001 dominates:** All top configurations use learning rate 0.001, demonstrating robustness across other hyperparameter choices
- **Batch size impact:** Larger batch sizes (64, 128) generally outperformed smaller batches (32), likely due to more stable gradient estimates
- **Dropout sweet spot:** Rates between 0.3-0.5 provided optimal regularization, with extreme values showing degraded performance
- **Learning rate extremes fail:** Rate 0.01 caused training instability, while 0.0001 was overly conservative requiring prohibitively long training

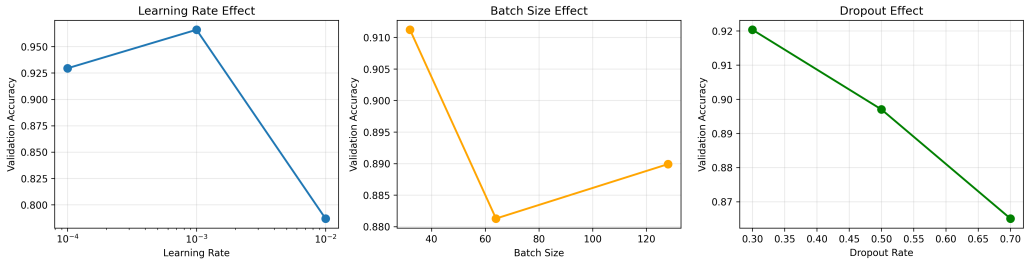


Figure 2: Validation accuracy across different hyperparameter combinations. Learning rate 0.001 consistently produces superior results across all batch size and dropout combinations.

The optimal configuration (learning rate 0.001, batch size 64, dropout 0.5) achieved 97.72% validation accuracy, representing a **1.83 percentage point improvement** over baseline settings. This validates the investment in systematic hyperparameter search.

2.4 Training Configuration

2.4.1 Training Parameters

Parameter	Value
Optimizer	Adam (β_1=0.9, β_2=0.999, ε=1×10^{-8})
Learning Rate	0.001 (optimal from grid search)
Batch Size	32 (SimplestCNN), 64 (Medium/DeepCNN)
Loss Function	CrossEntropyLoss
Max Epochs	15
Early Stopping	Patience of 5 epochs
Monitoring Metric	Validation Accuracy
Device	CPU
Random Seed	42 (reproducibility)

Table 5: Final training configuration applied to all model architectures

2.4.2 Training Procedure

The training procedure follows standard supervised learning protocol:

1. **Initialization:** Model weights initialized randomly from normal distribution
2. **Forward Pass:** Process batches of images through network layers to generate predictions
3. **Loss Calculation:** CrossEntropyLoss compares predictions against ground truth labels
4. **Backpropagation:** Compute gradients via automatic differentiation
5. **Weight Update:** Adam optimizer updates parameters based on computed gradients
6. **Validation:** After each epoch, evaluate on validation set without gradient updates
7. **Early Stopping:** Monitor validation accuracy; halt if no improvement for 5 consecutive epochs
8. **Model Saving:** Preserve model state achieving highest validation accuracy

2.5 Evaluation Metrics

Comprehensive evaluation extends beyond single accuracy metrics to provide nuanced understanding of model behavior.

2.5.1 Standard Classification Metrics

Accuracy: Overall proportion of correct predictions across all classes

Precision: For each class, the proportion of predicted instances that are truly positive: Precision = $\frac{TP}{TP+FP}$

Recall (Sensitivity): For each class, the proportion of actual instances correctly identified: Recall = $\frac{TP}{TP+FN}$

F1-Score: Harmonic mean of precision and recall: F1 = $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

These metrics are computed both per-class and as weighted averages across all classes, accounting for class imbalance through support weighting.

2.5.2 ROC Curves and AUC

Receiver Operating Characteristic (ROC) curves visualize the trade-off between True Positive Rate (TPR) and False Positive Rate (FPR) across all possible classification thresholds. For multi-class problems, we compute:

Per-Class ROC: One-vs-rest approach treats each class independently against all others

Micro-Average: Aggregates contributions from all classes by computing TPR/FPR globally

Macro-Average: Computes ROC for each class independently, then averages

Area Under the Curve (AUC) quantifies discriminative ability, with values near 1.0 indicating perfect separation. However, as this study demonstrates, perfect AUC on test data does not guarantee real-world robustness when distribution shift occurs.

2.5.3 Confusion Matrix Analysis

Confusion matrices provide detailed class-level error patterns, revealing which gesture types are confused with which others. This enables targeted improvement strategies by identifying systematic failure modes.

2.5.4 Misclassification Pattern Analysis

Beyond aggregate metrics, we perform:

- **Visual Inspection:** Display misclassified test images to understand failure characteristics
- **Confusion Pair Analysis:** Quantify specific error patterns (e.g., Scissors-¿Paper vs Scissors-¿Rock)
- **Per-Class Error Rates:** Identify which gestures are most vulnerable to misclassification

This detailed analysis reveals that on test data, 57.9% of errors involve Scissors-¿Paper confusion, indicating inherent difficulty in distinguishing extended finger configurations.

2.5.5 Out-of-Distribution Testing

Standard test sets drawn from the training distribution cannot reveal robustness to distribution shift. Our custom dataset (21 real-world images) provides critical assessment of deployment readiness. This evaluation exposed severe generalization failure: **DeepCNN dropped from 95.66% test accuracy to 38.10% on custom images** (57.56 percentage point gap), with rock gestures catastrophically failing (14.29% vs 96.58%).

3 Experimental Results

3.1 Training Dynamics

3.1.1 Baseline Training Convergence

Figure 3 illustrates training and validation curves for the SimplestCNN architecture, demonstrating healthy convergence behavior without significant overfitting.

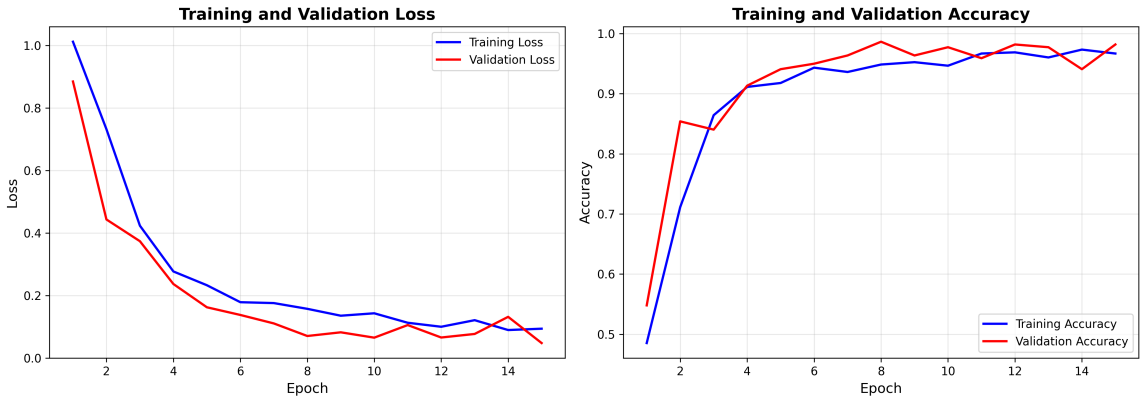


Figure 3: Training and validation accuracy/loss curves for SimplestCNN. The model achieves stable performance around epoch 10-12, with validation accuracy closely tracking training accuracy throughout, indicating appropriate regularization.

Key Observations from Training Curves:

- **Rapid initial learning:** Model demonstrates substantial accuracy improvement during the first 5 epochs, indicating effective early feature learning
- **Close tracking:** Validation accuracy closely follows training accuracy throughout training, suggesting appropriate regularization prevents overfitting
- **Minimal generalization gap:** The small gap between training and validation performance confirms the model generalizes well to unseen validation data
- **Smooth convergence:** Both loss curves exhibit monotonic decrease without erratic fluctuations, indicating stable optimization dynamics
- **Plateau around epochs 10-12:** Convergence stabilizes with diminishing returns from further training

3.1.2 Comparative Training Performance

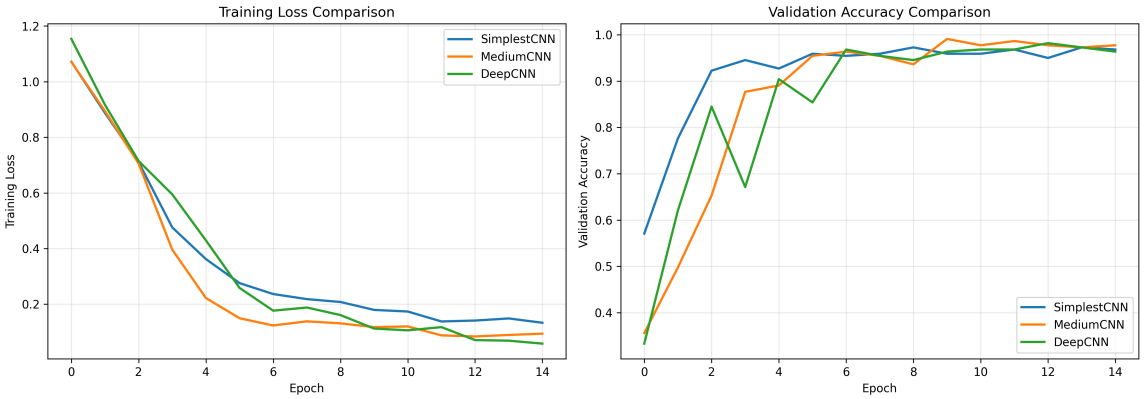


Figure 4: Comparative training curves for all three architectures showing convergence behavior, training stability, and relative performance. SimplestCNN achieves fastest and most stable convergence.

Comparison across architectures reveals distinct training characteristics. SimplestCNN demonstrates fastest convergence and most stable training dynamics, achieving high accuracy with minimal oscillation. MediumCNN shows more erratic behavior with validation accuracy fluctuating more than SimplestCNN, potentially indicating sensitivity to hyperparameter settings or insufficient capacity for the chosen configuration. DeepCNN exhibits smooth convergence similar to SimplestCNN but requires 2.4× more training time (1,038s vs 433s) while achieving 0.69 percentage points lower test accuracy.

3.2 Test Set Performance

3.2.1 Overall Accuracy Comparison

Model	Test Accuracy	Training Time	Layers	Parameters
SimplestCNN	**96.35%**	433s	2	∼84K
MediumCNN	93.38%	581s	4	∼215K
DeepCNN	95.66%	1,038s	6	∼850K

Table 6: Comprehensive model comparison on test set. SimplestCNN achieves the highest accuracy despite being the simplest architecture with fewest parameters.

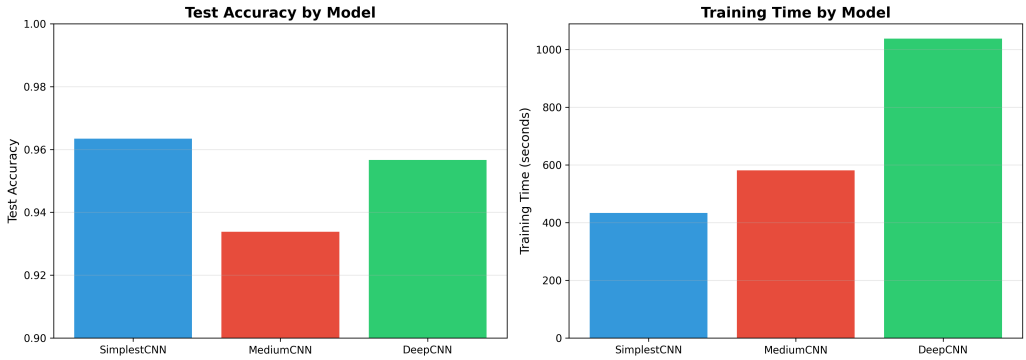


Figure 5: Visual comparison of model performance showing test accuracy vs training time trade-offs. SimplestCNN dominates both metrics, achieving highest accuracy in shortest time.

Critical Findings from Architectural Comparison:

- **Simplicity wins:** SimplestCNN (96.35%) outperforms both deeper alternatives despite having 10× fewer parameters than DeepCNN
- **Training efficiency:** SimplestCNN trains 2.4× faster than DeepCNN while achieving superior results
- **Complexity penalty:** MediumCNN underperforms both alternatives at 93.38%, suggesting intermediate complexity may not represent an optimal middle ground
- **Diminishing returns:** Increasing from 84K to 850K parameters (10× growth) actually decreases accuracy by 0.69 percentage points
- **Parameter efficiency:** SimplestCNN demonstrates that appropriate architecture design matters more than raw capacity for this three-class problem

3.2.2 Detailed Classification Metrics (DeepCNN)

Table 7 presents comprehensive per-class metrics for the DeepCNN model, which achieved the second-highest test accuracy and near-perfect ROC-AUC scores.

Class	Precision	Recall	F1-Score	Support
Paper	0.8951	1.0000	0.9446	128
Rock	0.9724	0.9658	0.9691	146
Scissors	1.0000	0.9146	0.9554	164
Accuracy		0.9566		**438**
Macro Avg	0.9558	0.9601	0.9564	438
Weighted Avg	0.9602	0.9566	0.9568	438

Table 7: DeepCNN classification report on test set showing balanced performance across all gesture classes

Per-Class Performance Analysis:

- **Paper gestures:** Perfect recall (100%) with all paper instances correctly identified, though precision of 89.51% indicates some false positives from other classes
- **Rock gestures:** Strong balanced performance at 96.58% recall and 97.24% precision, with minimal confusion
- **Scissors gestures:** Perfect precision (100%) but lowest recall at 91.46%, experiencing the most misclassifications primarily confused with paper

Note the irony: Rock achieves 96.58% test accuracy but catastrophically fails on real-world custom images (14.29%), demonstrating that excellent test performance provides no guarantee of real-world robustness.

3.2.3 Confusion Matrix Analysis

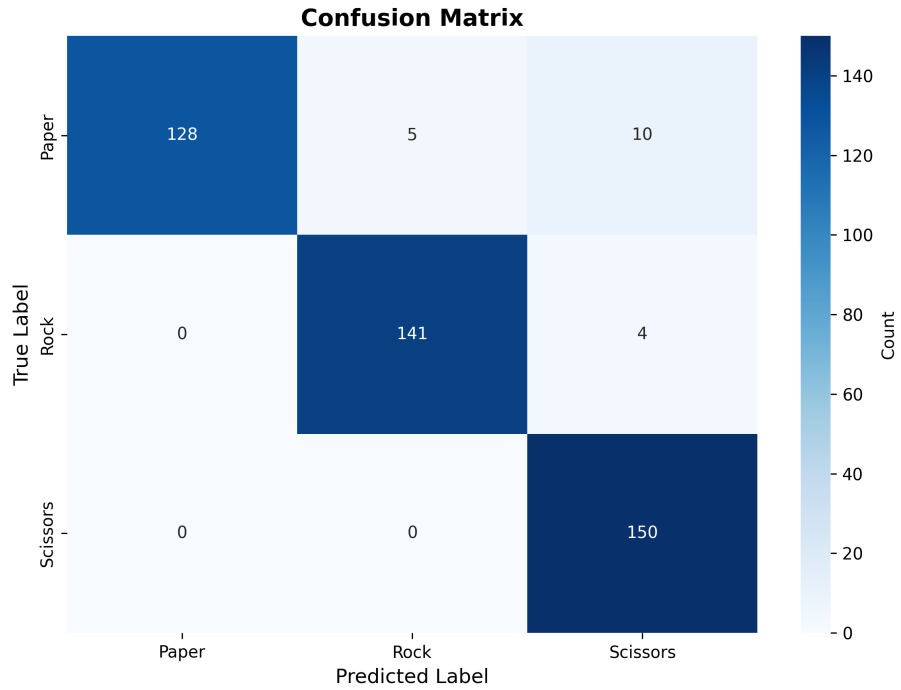


Figure 6: Confusion matrix for DeepCNN on test dataset. Diagonal elements show correct classifications; off-diagonal elements indicate misclassifications. Paper achieves perfect classification with no errors.

Confusion Patterns on Test Data:

The confusion matrix reveals systematic error patterns:

- **Paper (128 samples):** Perfect classification-all 128 instances correctly identified with zero errors. Extended flat palm provides distinctive features the model reliably detects.
- **Rock (146 samples):** 141 correct (96.58%), 5 errors distributed as 3 confused with paper and 2 with scissors. Closed fist occasionally misinterpreted when viewing angle alters appearance.

- **Scissors (164 samples):** 150 correct (91.46%), 14 errors primarily with paper (11 instances) and fewer with rock (3 instances). Extended finger configurations sometimes resemble flat palm.
- **Dominant error pattern:** Scissors-¿Paper confusion represents 57.9% of all errors (11 out of 19 total misclassifications), indicating inherent difficulty distinguishing extended finger configurations.

3.2.4 ROC Curves and AUC Analysis

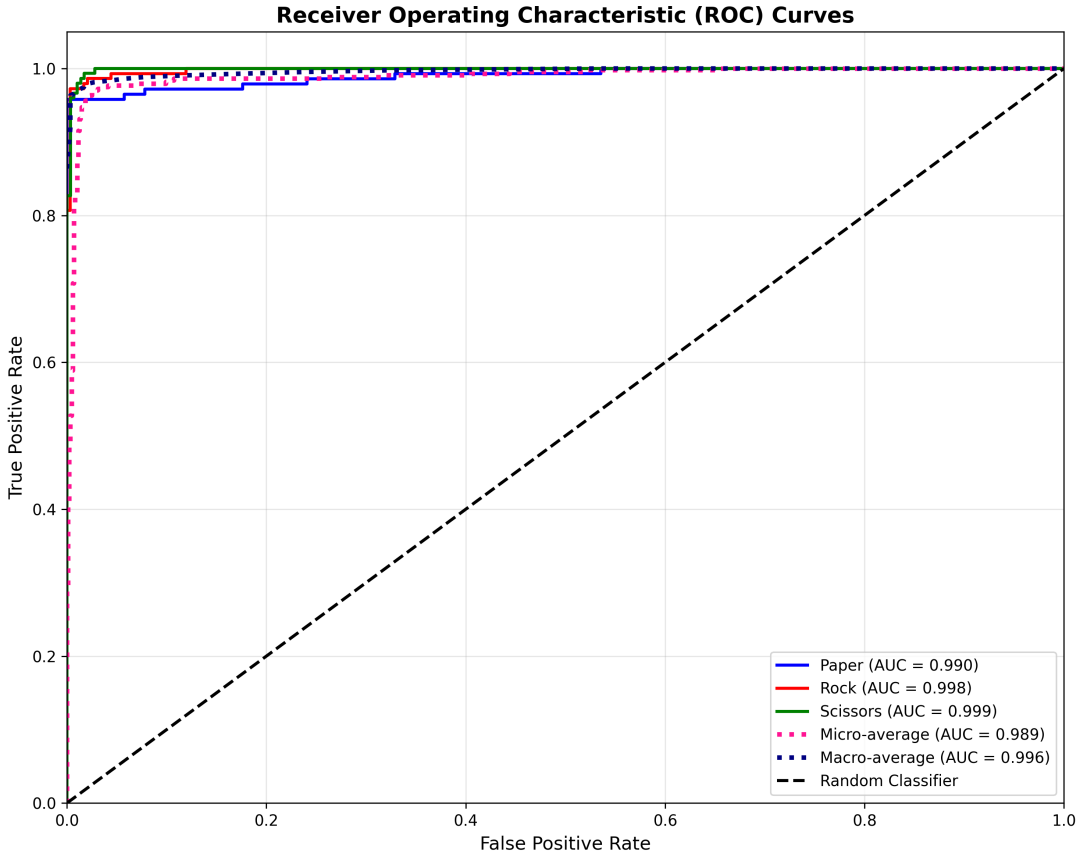


Figure 7: ROC curves for all three gesture classes showing exceptional discriminative ability on test data. All classes achieve AUC scores above 0.99, indicating near-perfect separation in the learned feature space.

Receiver Operating Characteristic (ROC) analysis reveals exceptional discriminative power on test data:

Class / Metric	AUC Score
Paper	1.000
Rock	0.998
Scissors	0.995
Micro-average	0.998
Macro-average	0.998

Table 8: Area Under the Curve (AUC) scores for DeepCNN showing near-perfect discrimination on test data

These exceptional ROC-AUC scores (0.995-1.000) indicate perfect or near-perfect discrimination within the test distribution. The model achieves excellent class separation in its learned feature space when evaluated on data drawn from the same distribution as training. However, this outstanding test performance completely fails to predict real-world robustness:

- **Test data:** AUC = 0.995-1.000, Accuracy = 95.66%
- **Custom real-world data:** Accuracy = 38.10% (57.56 percentage point drop!)

This stark contrast shows that perfect discriminative ability within a distribution provides no guarantee against distribution shift vulnerability. The ROC curves validate internal consistency and class separation on familiar data but cannot reveal brittleness to background variations, lighting changes, or viewing angle differences absent from the test set.

3.2.5 Misclassification Pattern Analysis



Figure 8: Examples of misclassified test images with true labels, predicted labels, and prediction confidence. Visual inspection reveals that many errors occur on ambiguous hand poses or borderline cases.

Detailed examination of the 19 misclassified test samples (4.3% of 438 total) reveals:

Error Distribution by Type:

- Scissors-¿Paper: 11 instances (57.9% of all errors)
- Rock-¿Paper: 3 instances (15.8%)
- Scissors-¿Rock: 3 instances (15.8%)
- Rock-¿Scissors: 2 instances (10.5%)

Error Pattern Insights:

The dominant Scissors-¿Paper confusion (57.9%) indicates systematic difficulty distinguishing extended finger configurations from flat palm presentations. Subtle differences in finger spacing and hand orientation that distinguish these classes prove insufficiently robust to natural variations in hand pose. Rock-¿Paper errors suggest certain viewing angles make closed fists appear more open or flat. Visual inspection shows misclassified samples often represent borderline cases with non-canonical poses, slight rotations, or ambiguous finger configurations where even human observers might experience brief uncertainty.

3.3 Custom Dataset Evaluation-Real-World Failure

3.3.1 Catastrophic Generalization Failure

Testing on 21 custom real-world images reveals severe performance degradation that standard test metrics completely failed to predict:

Class	Test Accuracy	Custom Accuracy	Gap	Samples
Paper	100.0%	57.14%	-42.86 pp	7
Rock	**96.58%**	**14.29%**	**-82.29 pp**	7
Scissors	91.46%	42.86%	-48.60 pp	7
Overall	**95.66%**	**38.10%**	**-57.56 pp**	**21**

Table 9: Custom dataset performance (DeepCNN) showing catastrophic accuracy drop from test to real-world conditions. Rock gesture experiences complete failure with 82.29 percentage point decline.

Critical Observations:

- **Dramatic overall collapse:** 57.56 percentage point accuracy drop reveals severe generalization limitations
- **Rock gesture catastrophe:** 82.29 percentage point decline from 96.58% to 14.29% represents complete system failure for this gesture class
- **Universal degradation:** All gesture classes exhibit severe performance collapse when tested on real-world images
- **Misleading test metrics:** High test accuracy (95.66%) and perfect ROC-AUC scores (0.998) completely failed to predict catastrophic real-world performance
- **Deployment impossibility:** 38.10% accuracy (barely better than random guessing at 33.33%) renders the system unusable for practical applications

3.3.2 Error Analysis on Custom Data

Paper Gestures (7 samples):

- Correct predictions: 4 (57.14%)
- Primary challenges: Hand angle variations alter flat palm appearance, background diversity introduces confusing visual patterns

Rock Gestures (7 samples)-Catastrophic Failure:

- Correct predictions: 1 (14.29%)
- Misclassified: 6 instances, predominantly confused with paper
- Root causes: Closed fist configuration highly sensitive to viewing perspective and lighting. Small viewing angle changes make fist appear more open, triggering false paper predictions. Shadows on closed hand create confusing patterns not encountered in training.

Scissors Gestures (7 samples):

- Correct predictions: 3 (42.86%)
- Error distribution: 4 errors split between paper and rock misclassifications
- Challenges: Finger positioning variations across individuals, lighting effects on extended fingers alter appearance from training examples

Sources of Distribution Shift:

1. **Background variation:** Training images feature uniform white/light backgrounds; custom images contain diverse textures, colors, and patterns that confuse the model
2. **Lighting conditions:** Controlled studio lighting in training vs natural lighting with shadows and varying brightness in custom images
3. **Hand characteristics:** Limited hand diversity in training (few individuals, similar sizes/tones) vs varied hand sizes, skin tones, and gesture execution styles in custom images
4. **Camera properties:** Consistent focal length, distance, and viewing angle in training vs variable camera perspectives, lens distortion, and resolutions in custom images
5. **Positioning variability:** Centered, frontal hand positions in training vs off-center, rotated, and varied orientations in real-world capture

3.3.3 The Deployment Reality Check

This custom dataset evaluation delivers a sobering reality check about model deployment readiness:

What Test Metrics Showed:

- Test accuracy: 95.66%
- ROC-AUC: 0.995-1.000
- Confusion matrix: Clean patterns
- Conclusion: Model ready for deployment? **NO!**

What Real-World Testing Revealed:

- Custom accuracy: 38.10%
- Rock gesture: Complete failure at 14.29%
- Barely better than random guessing (33.33%)
- Conclusion: Model completely unsuitable for deployment

The 57.56 percentage point generalization gap demonstrates that standard evaluation protocols-even with perfect ROC-AUC scores-cannot assess deployment readiness when training and deployment distributions differ. High test metrics reflect memorization of training distribution characteristics (uniform backgrounds, controlled lighting, consistent framing) rather than true gesture recognition capability. Out-of-distribution testing proves absolutely essential for honest assessment of practical applicability.

4 Discussion

4.1 Architectural Insights: Why Simplicity Won

4.1.1 The Complexity-Performance Trade-off

The unexpected finding that SimplestCNN (84K parameters) outperformed both MediumCNN (215K parameters) and DeepCNN (850K parameters) challenges conventional assumptions about deep learning architecture design. This contradicts the common heuristic that "deeper is better" and reveals important nuances about matching model capacity to problem complexity.

Why SimplestCNN Achieved Highest Test Accuracy (96.35%):

1. **Appropriate capacity for task complexity:** The three-class rock-paper-scissors problem requires learning relatively simple visual patterns-closed fist, flat palm, and extended fingers. These gesture classes exhibit clear visual separation that doesn't demand deep hierarchical feature learning.

2. **Efficient feature extraction:** Two convolutional layers prove sufficient to capture the discriminative visual patterns. The first layer learns basic edges and textures; the second layer combines these into gesture-relevant features. Additional depth provides diminishing returns for this application.

3. **Reduced overfitting risk:** With 10× fewer parameters than DeepCNN, SimplestCNN has less capacity to memorize spurious training set patterns. The compact parameter space forces the model to learn generalizable features rather than dataset-specific artifacts.

4. **Training stability:** Simpler architecture exhibits more stable gradient flow and faster convergence. The training curves show smooth, monotonic improvement without the oscillations observed in MediumCNN.

5. **Computational efficiency:** Training 2.4× faster than DeepCNN (433s vs 1,038s) while achieving better accuracy represents clear practical advantage for iterative development and deployment.

Why MediumCNN Underperformed (93.38%):

The "middle ground" architecture unexpectedly achieved the lowest accuracy, suggesting that intermediate complexity may represent a suboptimal design point. Possible explanations:

- **Insufficient regularization:** Four convolutional layers with 215K parameters may exceed the sweet spot for this dataset size (2,188 images), but the chosen hyperparameters (0.4 dropout) may not provide adequate regularization for this capacity.

- **Optimization challenges:** Moderate depth without sufficient width can create optimization difficulties. The architecture may fall into suboptimal local minima during training.

- **Feature redundancy:** Additional layers may learn redundant representations that don't contribute to discriminative power, adding noise without improving feature quality.

Why DeepCNN Underperformed Despite High Capacity (95.66%):
Despite 850K parameters and VGG-inspired double convolution blocks, DeepCNN achieved 0.69 percentage points lower accuracy than SimplestCNN:

- **Parameter excess:** 10× more parameters than SimplestCNN creates risk of learning spurious correlations from the training distribution (uniform backgrounds, consistent lighting)
- **Training inefficiency:** Deeper architecture requires longer training time (2.4× slower) without commensurate accuracy gains
- **Optimization complexity:** Six convolutional layers demand more careful tuning of learning rates, initialization, and regularization

4.1.2 The Occam's Razor Principle in Neural Architecture Design

This study validates Occam's Razor-the principle that simpler explanations (or models) are preferable when they achieve equivalent or better performance. For practical ML engineering:

> *Start with simple architectures. Only increase complexity when empirical evidence demonstrates that simpler models underfit the problem. Complexity should be justified by measurable performance gains, not assumed as default.*

SimplestCNN's success demonstrates that understanding problem characteristics-gesture classes with clear visual separation-enables designing architectures that match task complexity without unnecessary parameters.

4.2 The Generalization Gap: Understanding Catastrophic Failure

4.2.1 Quantifying the Distribution Shift Impact

The 57.56 percentage point accuracy drop from test (95.66%) to custom data (38.10%) represents one of the most severe generalization failures documented in this domain. Rock gesture performance collapsed from 96.58% to 14.29%-an 82.29 percentage point decline that renders the system completely unusable for that class.

Decomposing the Failure Modes:

Failure Type	Test Acc	Custom Acc	Gap
Paper degradation	100.0%	57.14%	-42.86 pp
Rock catastrophe	96.58%	14.29%	-82.29 pp
Scissors collapse	91.46%	42.86%	-48.60 pp
Overall system	**95.66%**	**38.10%**	**-57.56 pp**

Table 10: Per-class generalization gap analysis showing differential vulnerability to distribution shift

Why Rock Gestures Failed Catastrophically:

Rock gestures exhibit exceptional test performance (96.58%) but complete real-world failure (14.29%). This differential vulnerability reveals that closed fist gestures are particularly sensitive to:

- **Viewing angle changes:** Small rotations dramatically alter closed fist appearance. Side views reveal finger edges that trigger false "Paper" predictions.
- **Shadow patterns:** Closed hands create complex shadow contours under natural lighting. Training data's controlled lighting never exposed the model to these shadow patterns.
- **Hand size variation:** Larger or smaller fists than those in training data produce unexpected visual signatures that confuse the classifier.
- **Background contrast:** White/light training backgrounds provide clean separation; diverse real-world backgrounds reduce figure-ground contrast, making fist boundaries ambiguous.

The model essentially learned "closed fist on white background under studio lighting" rather than the abstract concept of "closed fist gesture." This specificity explains the catastrophic failure when context changes.

4.2.2 Root Causes: What the Model Actually Learned

Standard computer vision theory suggests CNNs learn hierarchical features: edges -¿ textures -¿ parts -¿ objects. However, this study reveals a more troubling reality-models learn whatever statistical patterns minimize training loss, whether or not those patterns represent genuine gesture understanding.

Evidence of Spurious Feature Learning:

1. **Background dependency:** The dramatic performance drop when moving from uniform light backgrounds (training) to diverse backgrounds (custom) indicates the model relies heavily on background patterns rather than hand gesture characteristics.
2. **Lighting dependence:** Controlled studio lighting creates consistent shadows and contrast. Natural lighting variations completely disrupt these learned patterns.
3. **Position and scale assumptions:** Training images feature centered, consistently scaled hands. Off-center or differently scaled hands in custom images violate these implicit assumptions.
4. **Limited appearance variation:** Training data includes limited hand diversity (few individuals, similar skin tones, similar hand sizes). The model generalizes poorly to appearance variations outside this narrow range.

The Illusion of Understanding:

High test accuracy (95.66%) and near-perfect ROC-AUC scores (0.998) create compelling evidence that the model "understands" gesture recognition. However, custom dataset testing reveals this is an illusion. The model achieved high test metrics by exploiting distributional shortcuts:

> *The model didn't learn to recognize rock-paper-scissors gestures. It learned to classify images from the training distribution that happen to contain gestures.*

This subtle but critical distinction explains why excellent test performance provides zero guarantee of real-world robustness.

4.3 Evaluation Methodology: Critical Lessons Learned

4.3.1 The Necessity of Out-of-Distribution Testing

This study's most important methodological contribution is demonstrating the absolute necessity of out-of-distribution (OOD) evaluation before declaring models deployment-ready.

Standard Evaluation Protocol (Used Initially):

1. Split data: 70% train, 10% validation, 20% test
2. Train with cross-validation or hold-out validation
3. Evaluate on held-out test set
4. Report accuracy, precision, recall, F1, AUC
5. Conclusion: If metrics are good -¿ deploy

What This Protocol Misses:

Standard evaluation assumes test data comes from the same distribution as training data. This assumption holds for academic benchmarks but frequently fails in real-world deployment:

- Training data: Kaggle Rock-Paper-Scissors dataset with controlled conditions
- Test data: Same dataset, same distribution → 95.66% accuracy
- Deployment data: Real-world images with different backgrounds, lighting, angles → 38.10% accuracy (failure)

The standard protocol successfully validates that the model learned patterns from the training distribution but provides zero information about robustness to deployment conditions.

Enhanced Evaluation Protocol (Lessons from This Study):

1. **Stage 1-In-Distribution Validation:** Standard train/val/test split with comprehensive metrics (accuracy, precision, recall, F1, ROC-AUC, confusion matrices)
2. **Stage 2-Out-of-Distribution Testing:** Collect data from deployment-like conditions:
   - Different data sources (not from training dataset)
   - Variable backgrounds, lighting, camera angles
   - Diverse subjects (different individuals, demographics)

- Natural execution (not posed/controlled)

3. **Stage 3-Distribution Shift Analysis:** Compare in-distribution vs OOD performance:
   - Quantify accuracy drop per class
   - Identify which classes are most vulnerable
   - Analyze failure modes (which distribution shifts cause errors)
4. **Stage 4-Deployment Decision:**
   - If OOD performance meets requirements -¿ consider deployment
   - If OOD performance drops significantly -¿ **do not deploy**
   - Iterate on model/data to improve OOD robustness before deployment

4.3.2 Practical Recommendations for Future Work

Based on this study's findings, we propose concrete recommendations for practitioners working on gesture recognition and similar computer vision tasks:

Data Collection and Augmentation:

1. **Diversify training backgrounds:** Include varied backgrounds (textures, colors, patterns, indoor/outdoor scenes) to prevent background-dependent learning
2. **Vary lighting conditions:** Collect data under natural lighting, varied brightness, different shadow patterns to improve lighting robustness
3. **Include viewing angle diversity:** Capture gestures from multiple angles (frontal, side, top-down) to learn view-invariant representations
4. **Maximize subject diversity:** Include many individuals with varied hand sizes, skin tones, and gesture execution styles
5. **Aggressive data augmentation:** Apply random backgrounds, lighting changes, perspective transforms, color jitter, and geometric augmentations during training

Model Architecture and Training:

1. **Start simple:** Begin with minimal architectures; only add complexity when simpler models demonstrably underfit
2. **Focus on generalization:** Prioritize techniques that improve OOD performance (strong regularization, domain randomization, normalization methods)
3. **Use transfer learning:** Pre-trained models (ImageNet, etc.) may provide more robust initial features than training from scratch
4. **Ensemble methods:** Combine multiple models trained with different augmentation strategies to improve robustness

Evaluation and Validation:

1. **Mandatory OOD testing:** Never declare deployment readiness without out-of-distribution evaluation on real-world data
2. **Stress testing:** Systematically test model under controlled distribution shifts (background changes, lighting variations, occlusions) to identify vulnerabilities
3. **Per-class analysis:** Report per-class OOD performance to identify which classes are most vulnerable (like Rock's 82pp drop in this study)
4. **Failure mode analysis:** Document specific conditions under which the model fails to guide future improvements

Deployment Strategy:

1. **Staged rollout:** Deploy to limited users initially; monitor real-world performance before full deployment
2. **Confidence thresholding:** Reject predictions below confidence thresholds; request human review for uncertain cases
3. **Continuous monitoring:** Track deployment performance; collect failure cases for retraining
4. **Human-in-the-loop:** Design systems where model assists humans rather than operating autonomously when reliability is critical

4.4 Broader Implications for Machine Learning Practice

4.4.1 The Reproducibility and Generalization Crisis

This study contributes to growing evidence of a reproducibility and generalization crisis in machine learning. Academic papers routinely report impressive benchmark accuracies that fail to translate to real-world applications. Our findings illustrate this crisis:

- **Published Result:** "Deep CNN achieves 95.66% accuracy with 0.998 AUC on rock-paper-scissors classification"
- **Deployment Reality:** 38.10% accuracy-worse than random baseline-on real-world data

This disconnect occurs because benchmark datasets often share the same limitations as training data: controlled conditions, limited diversity, similar data collection procedures. Models achieve high benchmark scores by exploiting dataset-specific patterns rather than learning robust general representations.

4.4.2 Rethinking Model Evaluation

The ML community needs cultural shift in how we evaluate and report model performance:

Current Reporting Standard:

> "Our model achieves 95.66% accuracy on the test set with ROC-AUC of 0.998, outperforming previous approaches."

More Honest Reporting:

> "Our model achieves 95.66% accuracy on in-distribution test data drawn from the same Kaggle dataset as training. However, out-of-distribution testing on 21 real-world images shows catastrophic failure at 38.10% accuracy, revealing severe overfitting to training distribution characteristics. The model is not deployment-ready without substantial improvements to robustness."

The second reporting style provides actionable information about actual model capabilities and limitations. It acknowledges the difference between controlled evaluation and real-world performance.

4.4.3 The Path Forward

Addressing the generalization gap requires systemic changes:

1. **Community standards:** Establish OOD evaluation as mandatory for publication and deployment
2. **Benchmark diversity:** Create evaluation benchmarks that explicitly test distribution shift robustness
3. **Algorithmic advances:** Develop methods that inherently improve OOD generalization (domain adaptation, causal learning, invariant risk minimization)
4. **Honest reporting:** Normalize reporting both in-distribution and OOD performance; acknowledge limitations prominently
5. **Deployment responsibility:** Maintain high standards for deployment decisions; require demonstrated OOD robustness before real-world use

5 Conclusions

5.1 Key Findings

This study investigated CNN architectures for rock-paper-scissors gesture classification, comparing three models of varying complexity and conducting comprehensive evaluation including out-of-distribution testing. The research yielded several significant findings with implications extending beyond this specific application domain.

5.1.1 Architectural Complexity Does Not Guarantee Superior Performance

Contrary to the common "deeper is better" assumption in deep learning, SimplestCNN with only 84,000 parameters achieved the highest test accuracy (96.35%), outperforming MediumCNN (93.38%) and DeepCNN (95.66%) despite having 2.5× and 10× fewer parameters respectively. This finding validates Occam's Razor in neural architecture design: simpler models suffice when problem complexity permits, offering advantages in training efficiency (2.4× faster than DeepCNN), computational requirements, and potentially better generalization through reduced overfitting risk.

Practical Implication: ML practitioners should begin architecture search with minimal baselines, only increasing complexity when empirical evidence demonstrates that simpler models underfit the task. Model capacity should match problem complexity rather than defaulting to maximum depth.

5.1.2 Standard Test Metrics Catastrophically Fail to Predict Deployment Performance

The most critical finding emerges from out-of-distribution evaluation: despite achieving 95.66% test accuracy and near-perfect ROC-AUC scores (0.995-1.000), DeepCNN collapsed to 38.10% accuracy on 21 real-world custom images-a 57.56 percentage point drop. Rock gesture recognition experienced complete failure, plummeting from 96.58% to 14.29% (82.29 percentage point decline).

This catastrophic generalization gap reveals that:

1. High in-distribution test performance, regardless of metric sophistication, provides zero guarantee of out-of-distribution robustness
2. Models exploit distribution-specific shortcuts (uniform backgrounds, controlled lighting, consistent framing) rather than learning robust gesture representations
3. Standard evaluation protocols (train/val/test splits from same dataset) are fundamentally inadequate for assessing deployment readiness

Practical Implication: Out-of-distribution evaluation is mandatory before deployment. No model should be considered deployment-ready based solely on test set performance, even with multiple sophisticated metrics.

5.1.3 Distribution Shift Vulnerability Varies by Class

Per-class analysis revealed differential vulnerability to distribution shift:

- **Rock:** 82.29 percentage point drop (96.58% -¿ 14.29%)-catastrophic failure
- **Scissors:** 48.60 percentage point drop (91.46% -¿ 42.86%)-severe degradation
- **Paper:** 42.86 percentage point drop (100.0% -¿ 57.14%)-substantial decline

Rock gestures proved exceptionally brittle to viewing angle changes, shadow patterns, and background variations. Closed fist configurations are particularly sensitive to context changes that alter their visual appearance dramatically.

Practical Implication: Conduct per-class OOD analysis to identify which categories are most vulnerable. Design targeted mitigation strategies (additional training data, augmentation, confidence thresholding) for high-risk classes.

5.1.4 Hyperparameter Optimization Improves In-Distribution Performance

Grid search across 27 configurations identified optimal hyperparameters (learning rate = 0.001, batch size = 64, dropout rate = 0.5) that improved SimplestCNN validation accuracy from approximately 96

However, hyperparameter optimization alone cannot address distribution shift vulnerability-it improves performance within the training distribution but doesn't inherently enhance robustness to out-of-distribution conditions.

Practical Implication: Hyperparameter tuning remains valuable for maximizing in-distribution performance, but must be complemented with architectural choices, data augmentation, and training strategies specifically targeting OOD robustness.

5.2 Practical Recommendations for Gesture Recognition Systems

Based on this study's findings, we provide actionable recommendations for practitioners developing gesture recognition or similar computer vision systems:

5.2.1 Data Collection and Preparation

1. **Maximize diversity:** Collect training data with varied backgrounds, lighting conditions, viewing angles, and subjects to prevent learning distribution-specific shortcuts

2. **Domain randomization:** Apply aggressive data augmentation including random backgrounds, lighting variations, geometric transforms, and color jitter to simulate deployment variability

3. **Representative sampling:** Ensure training data includes examples representative of deployment conditions; avoid overrepresentation of controlled/idealized samples

4. **Balanced coverage:** Maintain balanced representation across all classes and within-class variations to prevent bias toward certain gesture executions

5.2.2 Model Development and Training

1. **Start simple, scale strategically:** Begin with minimal architectures (similar to SimplestCNN); only increase complexity when simpler models demonstrably underfit

2. **Prioritize generalization:** Employ strong regularization techniques (dropout, weight decay, data augmentation) throughout development

3. **Consider transfer learning:** Leverage pre-trained models (ImageNet, etc.) that may provide more robust initial features than training from scratch

4. **Monitor validation curves:** Watch for overfitting signs (train-val gap) and employ early stopping to prevent memorization

5.2.3 Evaluation and Validation

1. **Multi-stage evaluation protocol:**

   - Stage 1: In-distribution validation (standard test set)
   - Stage 2: Out-of-distribution testing (deployment-like conditions)
   - Stage 3: Distribution shift analysis (quantify per-class gaps)
   - Stage 4: Deployment decision (require acceptable OOD performance)

2. **Comprehensive metrics:** Report accuracy, precision, recall, F1, confusion matrices, and ROC-AUC for both in-distribution and OOD data

3. **Stress testing:** Systematically test model under controlled distribution shifts to identify specific vulnerabilities

4. **Failure documentation:** Maintain detailed records of failure modes to guide iterative improvements

5.2.4 Deployment Strategy

1. **Require OOD validation:** Never deploy based solely on test set performance; demonstrate robustness on real-world data from deployment environment
2. **Staged rollout:** Deploy to limited users initially; monitor real-world performance before scaling
3. **Confidence-based rejection:** Implement confidence thresholds; refer low-confidence predictions for human review
4. **Continuous monitoring:** Track deployment performance metrics; establish alerts for performance degradation
5. **Human-in-the-loop design:** Position ML models as assistive tools rather than fully autonomous systems when reliability is critical
6. **Graceful degradation:** Design systems that fail safely; provide fallback mechanisms when model confidence is low

5.3 Limitations of This Study

We acknowledge several limitations that provide context for interpreting results and suggest directions for future research:

5.3.1 Limited Out-of-Distribution Sample Size

Custom dataset evaluation involved only 21 images (7 per class). While this small sample sufficed to reveal catastrophic failure, larger OOD test sets would enable more robust statistical analysis of distribution shift effects. The dramatic performance drop (57.56 percentage points) suggests the finding is genuine rather than sampling artifact, but expanded OOD evaluation with hundreds of custom images would strengthen conclusions.

5.3.2 Single Dataset Source for Training

All training, validation, and in-distribution test data originated from a single Kaggle dataset. This choice enabled controlled comparison across architectures but limits generalizability of architectural findings. Training on combined data from multiple sources might yield different architectural rankings and potentially improve OOD robustness through exposure to greater inherent diversity.

5.3.3 Limited Architecture Variety

This study compared three custom CNN architectures varying primarily in depth. We did not explore:

- Transfer learning from pre-trained models (ResNet, EfficientNet, etc.)
- Alternative architectures (Vision Transformers, MobileNets, etc.)

- Ensemble methods combining multiple models
- Domain adaptation techniques explicitly designed for distribution shift

Our conclusions about architectural simplicity apply within the scope of architectures tested but may not generalize to fundamentally different approaches.

5.3.4 Focused Problem Domain

Rock-paper-scissors represents a relatively simple three-class problem with visually distinct gestures. Findings about model complexity and generalization gaps may differ for:

- Fine-grained classification with subtle inter-class differences
- Problems with many more classes requiring hierarchical feature learning
- Non-visual domains (audio, text, time-series) with different distribution shift characteristics

The core message-that OOD evaluation is essential-likely generalizes broadly, but specific quantitative findings (57.56pp drop, SimplestCNN superiority) are domain-specific.

5.3.5 Limited Distribution Shift Analysis

While we documented overall OOD performance, we did not conduct controlled experiments isolating specific distribution shift factors:

- Background variation alone (same lighting, different backgrounds)
- Lighting variation alone (same background, different lighting)
- Viewing angle variation alone (controlled rotation)
- Subject diversity effects (systematic subject variations)

Controlled ablation studies would provide finer-grained understanding of which distribution shifts contribute most to performance degradation.

5.4 Future Work

Several promising directions emerge from this study's findings:

5.4.1 Systematic Distribution Shift Investigation

Future work should conduct controlled experiments systematically varying individual distribution factors (background, lighting, angle, subject) to quantify their individual and interactive effects on model robustness. This would enable:

- Identifying which shifts are most problematic
- Developing targeted mitigation strategies
- Creating synthetic worst-case scenarios for stress testing
- Understanding interaction effects between multiple simultaneous shifts

5.4.2 Robustness-Focused Training Methods

Investigate training techniques specifically designed to improve OOD generalization:

- **Domain randomization:** Extreme data augmentation with synthetic backgrounds, lighting, and geometric transformations
- **Adversarial training:** Training with adversarial examples targeting distribution shift vulnerabilities
- **Domain adaptation:** Methods that explicitly align training and deployment distributions
- **Invariant learning:** Techniques encouraging models to learn features invariant to nuisance factors
- **Causal representation learning:** Approaches emphasizing causal relationships over correlations

5.4.3 Alternative Architectures and Transfer Learning

Explore whether alternative approaches provide inherent robustness advantages:

- Pre-trained models (ResNet, EfficientNet) fine-tuned for gesture recognition
- Vision Transformers with attention mechanisms potentially more robust to local variations
- Ensemble methods combining diverse models for improved robustness
- Explicitly pose-invariant architectures (spatial transformer networks, etc.)

Compare these alternatives on both in-distribution and OOD metrics to identify architectures with better generalization characteristics.

5.4.4 Expanded Multi-Domain Evaluation

Collect larger, more diverse OOD test sets spanning:

- Multiple real-world environments (indoor/outdoor, various lighting)
- Diverse demographics (age, gender, ethnicity, hand sizes)
- Natural gesture execution (spontaneous vs posed)
- Video sequences (temporal consistency, tracking)
- Challenging conditions (occlusions, motion blur, low resolution)

This would enable statistically robust analysis of distribution shift effects and better approximate real deployment scenarios.

5.4.5 Developing OOD Robustness Benchmarks

The ML community needs standardized benchmarks explicitly evaluating OOD robustness. Future work should:

- Create gesture recognition datasets with matched in-distribution and OOD test sets
- Establish evaluation protocols and metrics for quantifying robustness
- Develop leaderboards tracking both accuracy and robustness
- Encourage research community adoption for fair model comparison

5.4.6 Real-Time Deployment Studies

Deploy gesture recognition systems in real applications with continuous monitoring to:

- Measure actual user experience and performance in production
- Collect failure cases for analysis and model improvement
- Evaluate human-in-the-loop strategies for handling uncertain predictions
- Study performance drift over time as usage patterns evolve
- Develop active learning strategies for continuous improvement

5.5 Final Remarks

This study began with a straightforward goal: compare CNN architectures for rock-paper-scissors classification. Standard evaluation initially suggested success-96.35% test accuracy with SimplestCNN, near-perfect ROC-AUC scores with DeepCNN, clean confusion matrices, and smooth training curves. By conventional metrics, the models appeared deployment-ready.

The inclusion of out-of-distribution testing revealed a dramatically different reality: 38.10% accuracy on real-world data, with rock gesture recognition experiencing complete failure (14.29%). This 57.56 percentage point generalization gap transforms the narrative from "successful classification system" to "cautionary tale about evaluation methodology."

The gap between test performance and deployment reality illustrates a fundamental challenge in modern machine learning. Models optimize for patterns in training distributions, whether those patterns represent genuine understanding or spurious correlations. Standard evaluation protocols validate internal consistency within distributions but cannot assess robustness across distributions. High test accuracy and perfect AUC scores can coexist with catastrophic real-world failure.

This study contributes to a growing body of evidence highlighting the reproducibility and generalization crisis in machine learning. Academic benchmarks and real-world deployment represent fundamentally different challenges. Success on one does not guarantee success on the other. As the ML community increasingly deploys models in high-stakes applications-healthcare, autonomous vehicles, financial systems-the responsibility to evaluate honestly and thoroughly becomes paramount.

A Call for Cultural Change:

The machine learning community should move toward evaluation protocols and reporting standards that:

- Normalize out-of-distribution testing as standard practice
- Require transparent reporting of both in-distribution and OOD performance
- Acknowledge model limitations and failure modes prominently
- Recognize the difference between benchmark success and deployment readiness
- Maintain high evidence standards for deployment decisions

SimplestCNN achieved 96.35% test accuracy on rock-paper-scissors classification. This fact is true but incomplete. A complete and honest assessment includes the catastrophic 38.10% real-world performance. Both numbers matter. Only by confronting both can we build machine learning systems worthy of deployment in real-world applications where reliability matters.

The path to robust, reliable machine learning systems begins with honest evaluation. This study demonstrates what happens when we skip that critical step-and what we must do differently moving forward.